

NATIONAL UNIVERSITY OF MODERN LANGUAGES

Program fundamental assignment # 03



SUBMITTED BY:

ZULQARNAIN HASSAN

SUBMITTED TO:

SIR AAQIB ADEEL

ROLL No:	SP21379
Faculty:	Engineering and computer science
Department:	Software engineering
Section:	BSSE-1-A-afternoon

Q No #01:- what is recursion? Explain with two programming examples.

Recursion:

A function that calls itself is known as a recursive function. And, this technique is known as recursion.

The recursion continues until some condition is met.

The Program below shows how recursion works by calling itself over and over again.

```
#include <iostream>

using namespace std;

int factorial(int);

int main()
{
    int n, result;

    cout << "Enter a non-negative number: ";

    cin >> n;

    result = factorial(n);

    cout << "Factorial of " << n << " = " << result;

    return 0;
}

int factorial(int n)
{
    if (n > 1)
    {
        return n * factorial(n - 1);
    } else
    {
        return 1;
    }
}
```

Output



E:\Untitled1.exe

```
Enter a non-negative number: 4
Factorial of 4 = 24
```

```
-----
```

```
Process exited after 4.512 seconds with return value 0
Press any key to continue . . .
```

```
#include <iostream>

using namespace std;

int fibonnaci(int x)
{
    if((x==1) || (x==0))
    {
        return(x);
    }
    else
    {
        return(fibonnaci(x-1)+fibonnaci(x-2));
    }
}

int main()
{
    int x , i=0;

    cout << "Enter the number of terms of series : ";

    cin >> x;

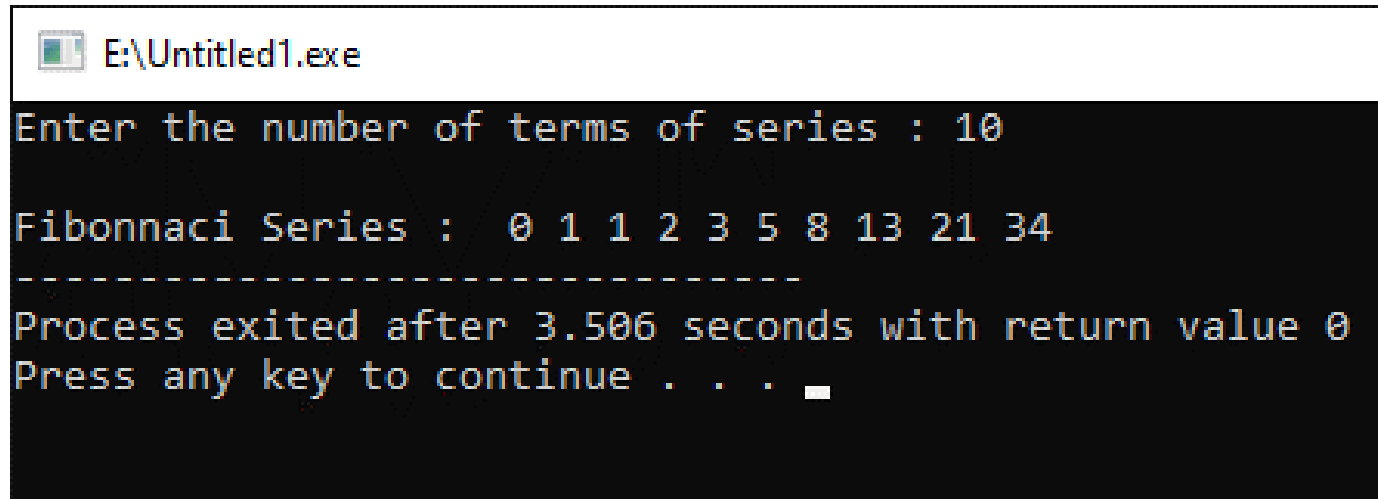
    cout << "\nFibonnaci Series : ";

    while(i < x)
    {
        cout << " " << fibonnaci(i);

        i++;
    }
}
```

```
}  
return 0;  
}
```

Output



```
E:\Untitled1.exe  
Enter the number of terms of series : 10  
Fibonnaci Series : 0 1 1 2 3 5 8 13 21 34  
-----  
Process exited after 3.506 seconds with return value 0  
Press any key to continue . . .
```

Q No #02:- explain various built-in string operations/functions through coding examples.

Strings

Strings are used for storing text. A string variable contains a collection of characters surrounded by double quotes:

Commonly used built-in string:

strxfrm()

The strxfrm() function in C++ transforms a given null-terminated byte string into an implementation-defined form.

Example

```
#include <iostream>  
#include <cstring>  
#include <clocale>  
using namespace std;  
int main()  
{  
    setlocale(LC_COLLATE, "cs_CZ.UTF-8");  
    const char* s1 = "hrnec";
```

```

const char* s2 = "chrt";
char t1[20], t2[20];
cout << "strcoll returned " << strcoll(s1,s2) << endl;
cout << "Before transformation, " << "strcmp returned " << strcmp(s1,s2) << endl;
strxfrm(t1,s1,10);
strxfrm(t2,s2,10);
cout << "After transformation, " << "strcmp returned " << strcmp(t1,t2) << endl;
}

```



E:\Untitled1.exe

```

strcoll returned 1
Before transformation, strcmp returned 1
After transformation, strcmp returned 1

-----
Process exited after 0.4635 seconds with return value 0
Press any key to continue . . .

```

strcoll()

The strcoll() function in C++ compares two null terminating string. The comparison is based on the current locale defined by the LC_COLLATE category.

strcmp() is enough for most string comparisons, but when dealing with unicode characters, sometimes there are certain nuances that make byte-to-byte string comparison incorrect.

```
#include <cstring>
```

```
#include <iostream>
```

```
using namespace std;
```

```
main()
```

```
{
```

```
    char lhs[] = "Armstrong";
```

```
    char rhs[] = "Army";
```

```
    int result;
```

```
    result = strcoll(lhs,rhs);
```

```

cout << "In the current locale ";

    if(result > 0)

        cout << rhs << " precedes " << lhs << endl;

    else if (result < 0)

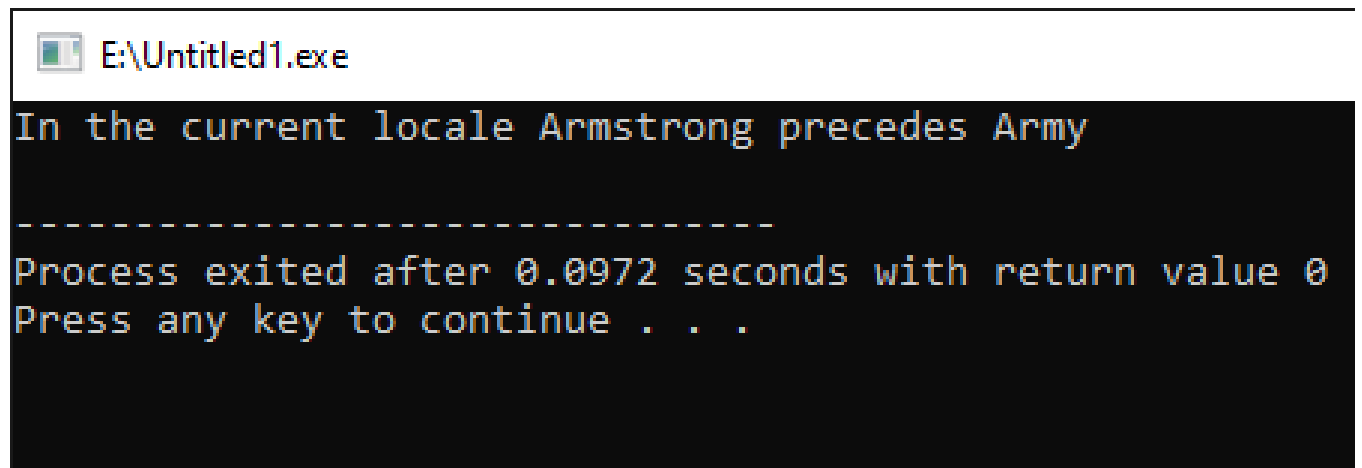
        cout << lhs << " precedes " << rhs << endl;

    else

        cout << lhs << " and " << rhs << " are same" << endl;

}

```



```

E:\Untitled1.exe
In the current locale Armstrong precedes Army
-----
Process exited after 0.0972 seconds with return value 0
Press any key to continue . . .

```

strlen()function:

The `strlen()` function in C++ returns the length of the given C-string. It is defined in the [cstring](#) header file.

```

#include <iostream>
#include <cstring>
using namespace std;

main() {
    char song[] = "We Will Rock You!";

    // print the length of the song string
    cout << strlen(song);
}

```

 E:\Untitled1.exe

17

Process exited after 0.1099 seconds with return value 0

Press any key to continue . . .

memset()

The memset() function in C++ copies a single character for a specified number of times to an object.

```
#include <cstring>
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    char dest[50];
```

```
    char ch = 'a';
```

```
    memset(dest, ch, 20);
```

```
    cout << "After calling memset" << endl;
```

```
    cout << "dest contains " << dest;
```

```
    return 0;
```

```
}
```

 E:\Untitled1.exe

```
After calling memset
dest contains aaaaaaaaaaaaaaaaaaaaaa
-----
Process exited after 0.185 seconds with return value 0
Press any key to continue . . .
```

strstr()

The strstr() function in C++ finds the first occurrence of a substring in a string.

```
#include <cstring>
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    char str[] = "Use your brain and heart";
```

```
    char target[] = "brain";
```

```
    char *p = strstr(str, target);
```

```
    if (p)
```

```
        cout << "\"" << target << "\" is present in \"" << str << "\" at position " << p-str;
```

```
    else
```

```
        cout << target << " is not present \"" << str << "\"";
```

```
    return 0;
```

```
}
```

E:\Untitled1.exe

```
'brain' is present in "Use your brain and heart" at position 9
-----
Process exited after 0.09164 seconds with return value 0
Press any key to continue . . .
```

strpbrk()

The strpbrk() function in C++ searches for a set of characters present in a string in another string.

```
#include <iostream>
```

```
#include <cstring>
```

```
#include <cstdio>
```

```
using namespace std;
```

```
main()
```

```
{
```

```
    char digits[] = "0123456789";
```

```
    char code[] = "ceQasieoLPqa4xz10lyq";
```

```
    char *pos;
```

```
    int count = 0;
```

```
        pos = strpbrk (code, digits);
```

```
    while (pos != NULL)
```

```
    {
```

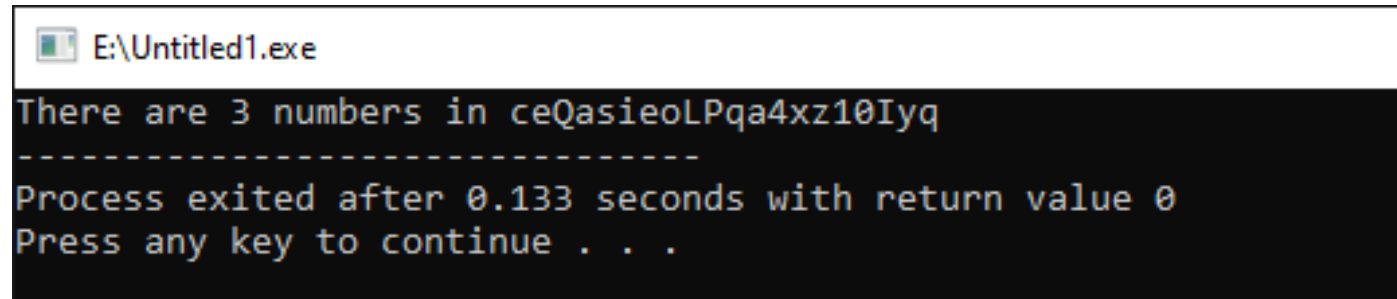
```
        pos = strpbrk (pos+1,digits);
```

```
        count ++;
```

```
    }
```

```
cout << "There are " << count << " numbers in " << code;

}
```



```
E:\Untitled1.exe
There are 3 numbers in ceQasieoLPqa4xz10Iyq
-----
Process exited after 0.133 seconds with return value 0
Press any key to continue . . .
```

strcmp()function:

The `strcmp()` function in C++ compares two null-terminating strings (C-strings). The comparison is done lexicographically. It is defined in the `cstring` header file.

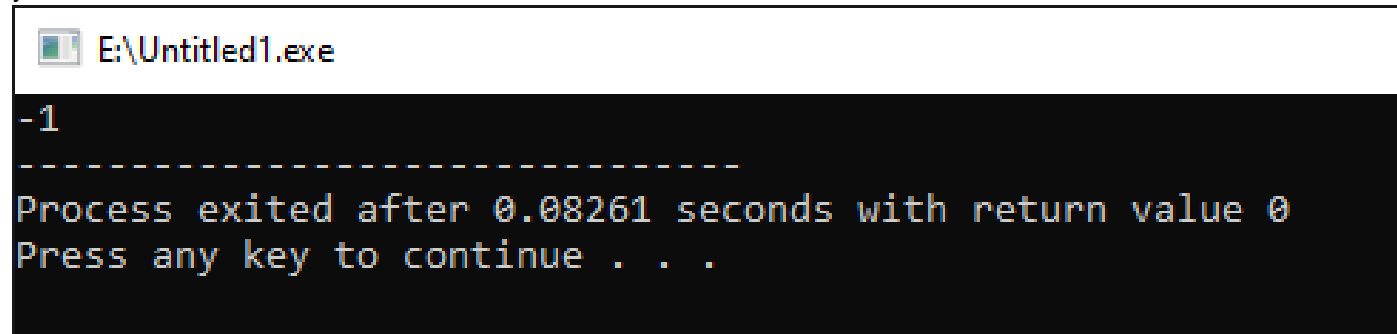
```
#include <cstring>
#include <iostream>
using namespace std;

int main() {
    char str1[] = "Megadeth";
    char str2[] = "Metallica";

    // compare str1 and str2 lexicographically
    int result = strcmp(str1, str2);

    cout << result;

    return 0;
}
```



```
E:\Untitled1.exe
-1
-----
Process exited after 0.08261 seconds with return value 0
Press any key to continue . . .
```

strchr()function:

The `strchr()` function in C++ searches for the first occurrence of a character in a string.

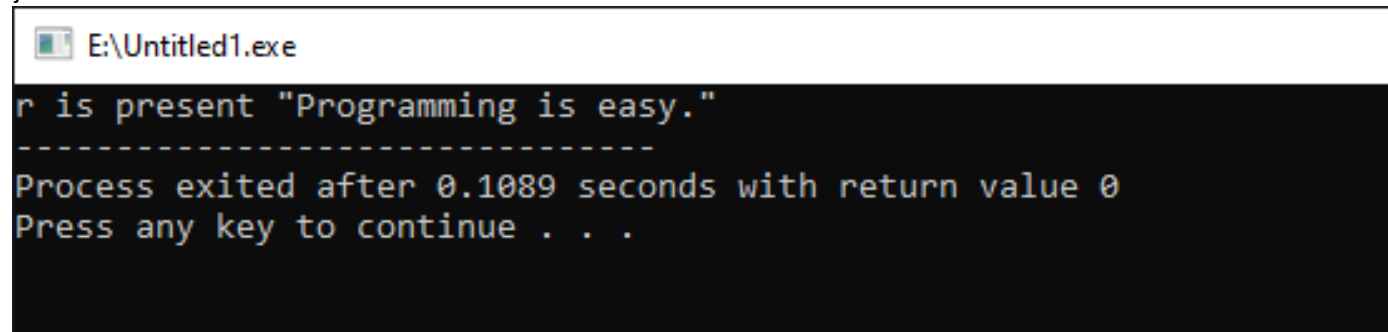
```
#include <cstring>
#include <iostream>

using namespace std;

int main()
{
    char str[] = "Programming is easy.";
    char ch = 'r';

    if (strchr(str, ch))
        cout << ch << " is present \\" << str << "\\";
    else
        cout << ch << " is not present \\" << str << "\\";

    return 0;
}
```

A screenshot of a Windows command prompt window. The title bar shows the file path "E:\Untitled1.exe". The command prompt displays the output of the program: "r is present \"Programming is easy.\"". Below this, there is a dashed line separator. The next line shows "Process exited after 0.1089 seconds with return value 0". The final line says "Press any key to continue . . .".

```
E:\Untitled1.exe
r is present "Programming is easy."
-----
Process exited after 0.1089 seconds with return value 0
Press any key to continue . . .
```

Strcpy()function:

The `strcpy()` function in C++ copies a character string from source to destination. It is defined in the `cstring` header file.

```
#include <cstring>
#include <iostream>

using namespace std;

int main() {
    char src[] = "Hello Programmers.";
    char dest[20];
    strcpy(dest,src);
    cout << dest;

    return 0;
}
```

 E:\Untitled1.exe

Hello Programmers.

Process exited after 0.07126 seconds with return value 0

Press any key to continue . . . █

strcat()function:

The strcat() function in C++ appends a copy of a string to the end of another string.

```
#include <cstring>
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    char dest[50] = "Learning C++ is fun";
```

```
    char src[50] = " and easy";
```

```
    strcat(dest, src);
```

```
    cout << dest ;
```

```
    return 0;
```

```
}
```

 E:\Untitled1.exe

Learning C++ is fun and easy

Process exited after 0.08215 seconds with return value 0

Press any key to continue . . . █

Q No #03:- What is sorting in an array? How does bubble sort work?

Sorting:

Sorting an array means arranging the elements in the array in a certain order. Various algorithms have been designed that sort the array using different methods. Some of these sorts are more useful than others in certain situations.

Here is a step-by-step breakdown of how bubble sorting works:

1. Start with an unsorted list

Bubble sorting starts with an unordered set of items. For example, imagine you are writing a code for an application and need to sort the following data set in ascending order: 4, 3, 1, 2, 5. Since this is a small data set that is mostly ordered already, you can decide to use bubble sort because it is a simple code to implement. After incorporating the code, the bubble sorting algorithm puts the numbers in order by comparing two elements at a time.

2. Compare the first two elements

The algorithm starts by comparing the first two items in the set. If the first item in the set is greater than the second item in the set, it swaps these items. For example, the first number in the data set above is four, which is greater than the second item in the data set, three. The algorithm swaps these numbers and the new data set becomes 3, 4, 1, 2, 5.

3. Assess the second and third elements

Next, the algorithm compares the second item in the list with the third item in the list. If the second item in the list is greater than the third item in the list, the algorithm swaps the numbers. Continuing with the sample data set, the second item in the set is the number four, which is greater than the third item in the set, the number one. The algorithm swaps these numbers and the new data set becomes 3, 1, 4, 2, 5.

4. Continue with the third and fourth elements

The algorithm continues the pattern. Next, it compares the third item in the data set with the fourth item in the data set. In the sample data set, the third number is the number four and the fourth number is two. Since four is greater than two, the algorithm again swaps the numbers. Now, the data set should be 3, 1, 2, 4, 5.

5. Complete the first pass and evaluate the list

Now, the algorithm compares the fourth item in the data set with the fifth item in the set. In the sample list, the fourth item is the number four and the fifth item is the number five. Since the number five is greater than the number four, the algorithm does not swap the items. The set stays 3, 1, 2, 4, 5, and the algorithm completes its first pass. However, since the list is still out of order, the algorithm must complete additional passes. The code automatically prompts the algorithm to continue the pattern until it sorts the entire set.

6. Continue making passes until fully sorted

The algorithm begins its second pass. It follows the same pattern listed in steps 1-5, comparing two items in the set at a time and swapping elements as needed. The algorithm performs as many passes as needed to arrange all items in the set in the correct order. Once

it recognizes that the order is correct, it stops without needing additional input. Since the algorithm stops once it has sorted the list, it is easy to know when the list is complete. When the algorithm stops, your set is ready for use.